

Projetos em Ambientes Web e Cloud

Licenciatura em Tecnologias Digitais e
Inteligência Artificial

Aula 5

João Matos

joao_miguel_goncalves_matos@iscte-iul.pt

2º Semestre

Exercício

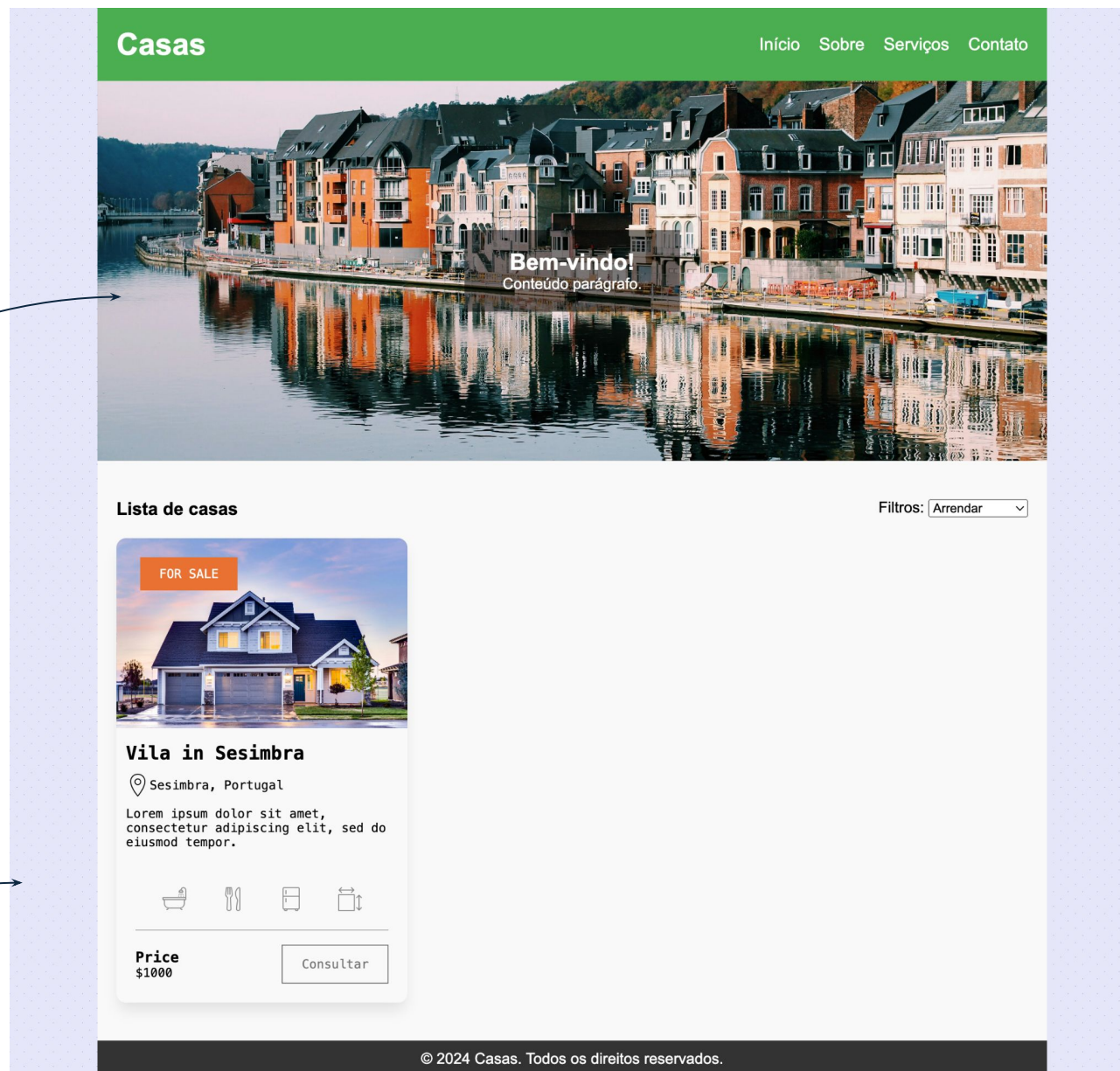
Home Page

- Container com header, main e footer, usando propriedade display: grid;
- Header color: #4CAF50
- Footer color: #333333

Section com background-image e elementos posicionados ao centro com Flexbox

body com background-image

max-width: 1000px

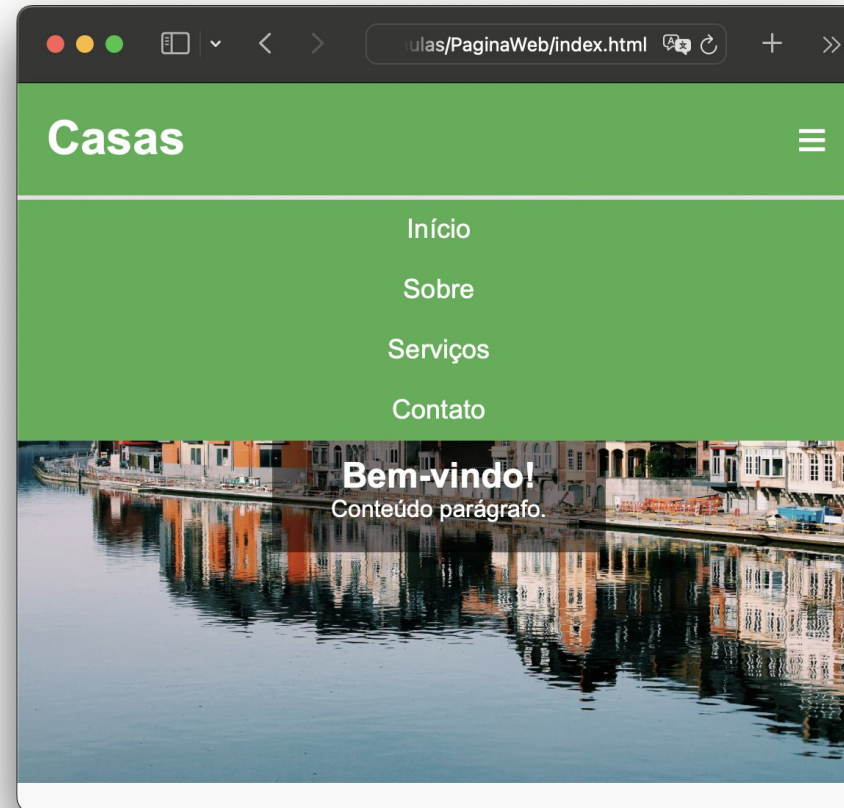


Exercício

<nav>, com class: “nav”
, com class: “nav-list”

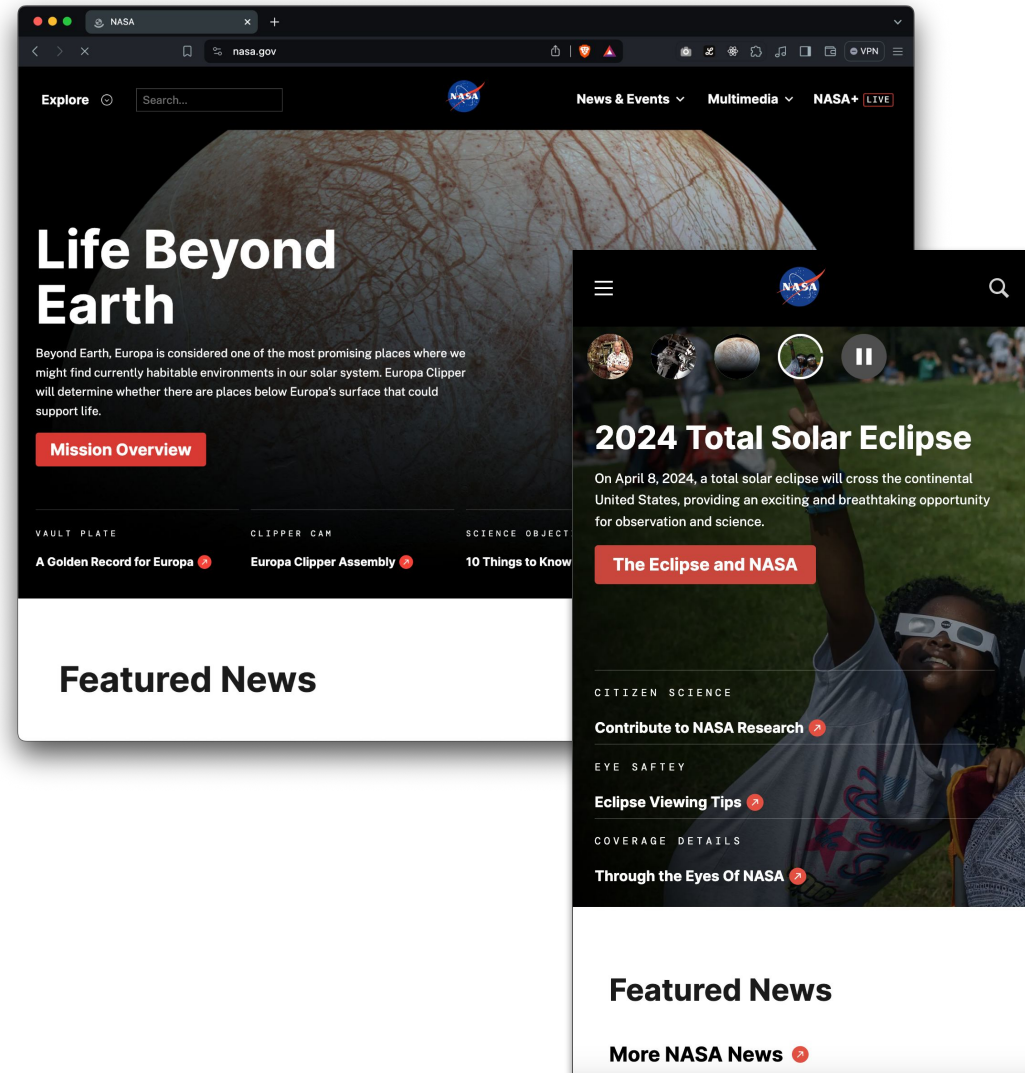
Incluir o seguinte <div> dentro do
<header>

```
<div class="menu-toggle" id="menu-toggle">  
  ≡  
</div>
```



Media Queries

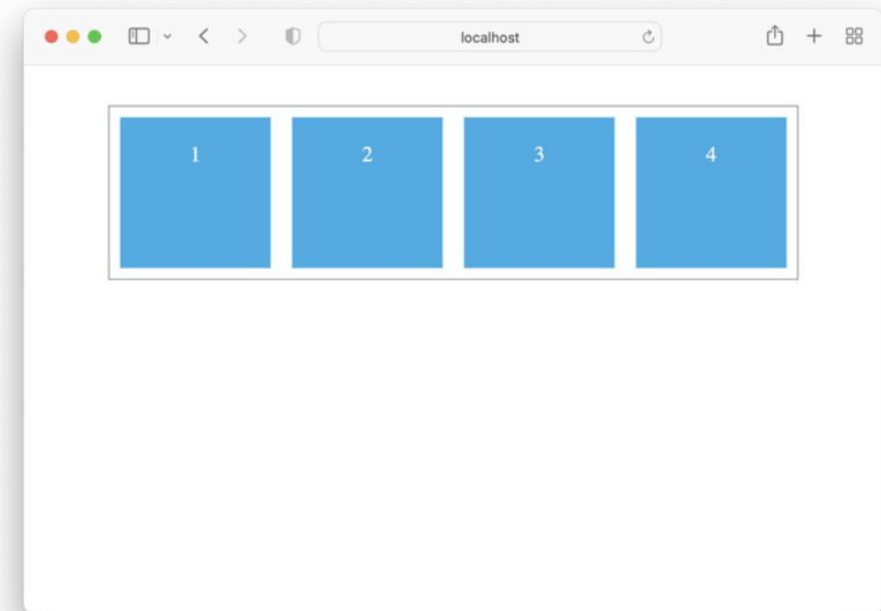
- Como podemos programar a nossa página para aparecer bem em todos os ecrãs?
- Utilizar larguras máximas não é suficiente e é necessário definir condições concretas para como o layout é apresentado em ecrãs de dimensões diferentes.



Media Queries

- É habitual que uma página apresente uma disposição horizontal de elementos em ecrãs mais largos.
- Como podemos alinhar aqueles elementos verticalmente quando o ecrã fica menos largo?

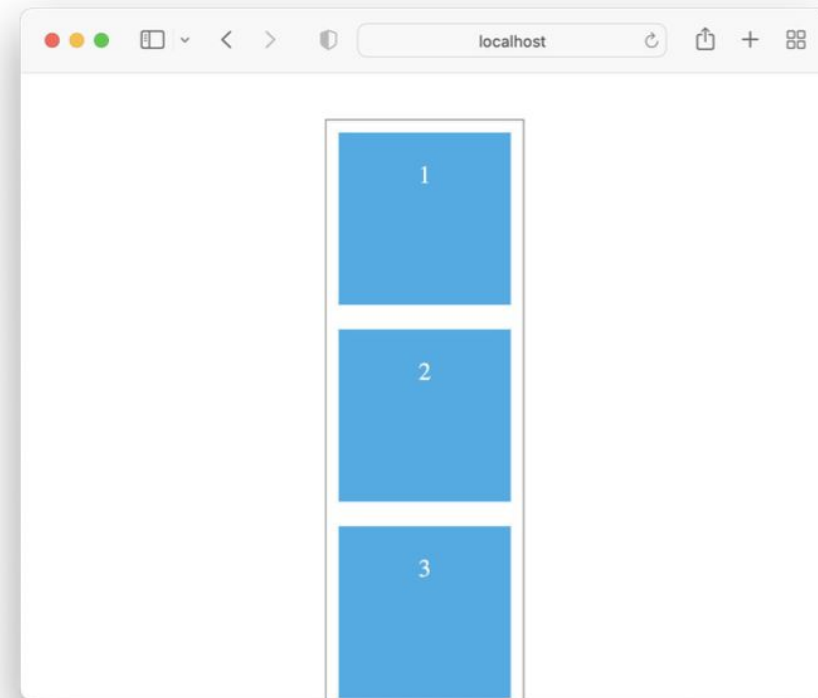
```
.container {  
  position: relative;  
  display: flex;  
  border: #8B8B8B solid 1px;  
  margin: 30px;  
  gap: 10px;  
  padding: 10px;  
}
```



Media Queries

```
.container {  
  position: relative;  
  display: flex;  
  border: #8B8B8B solid 1px;  
  margin: 30px;  
  gap: 10px;  
  padding: 10px;  
}  
  
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

width <= 768px



Media Queries

- Existe um conjunto universal de valores que definem as quebras nas páginas

Dispositivos muito pequenos (600px ou menos)	@media screen and (max-width: 600px) { . . . }
Dispositivos pequenos (600px ou mais)	@media screen and (min-width: 600px) { . . . }
Dispositivos médios (768px ou mais)	@media screen and (min-width: 768px) { . . . }
Dispositivos grandes (992px ou mais)	@media screen and (min-width: 992px) { . . . }
Dispositivos muito grandes (1200px ou mais)	@media screen and (min-width: 1200px) { . . . }

Media Queries

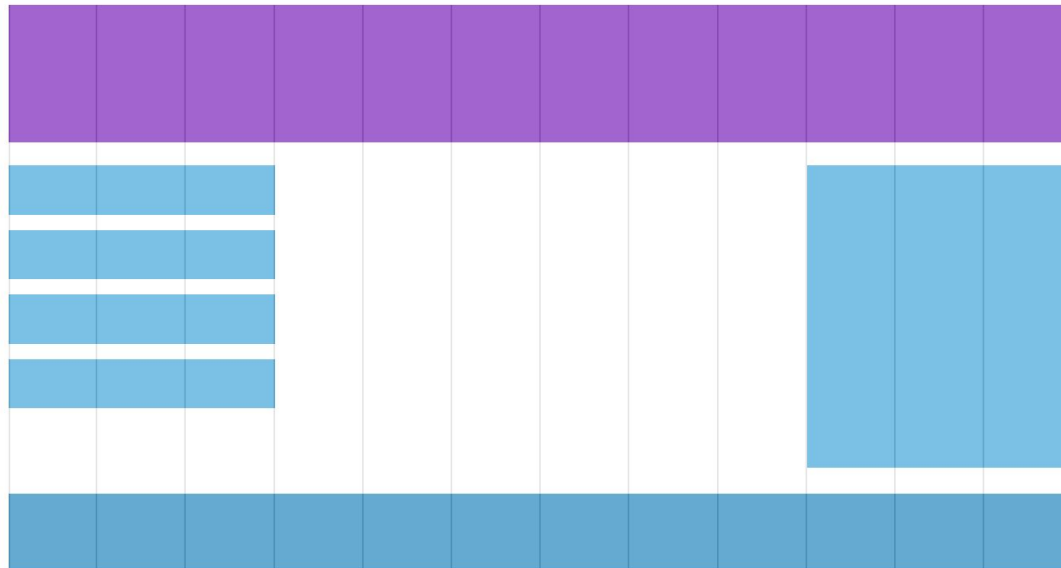
- Para a página ser corretamente apresentada nos dispositivos móveis é necessário adicionar a seguinte tag <meta> dentro do head:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- Caso esta linha não seja utilizada, o dispositivo vai considerar uma largura de janela maior e poderá não aplicar as regras de responsabilidade.

Media Queries

- Para simplificar o sistema, podemos dividir a página em colunas - **Grid-view**.



```
.col-1 {width: 8.33%;}  
.col-2 {width: 16.66%;}  
.col-3 {width: 25%;}  
.col-4 {width: 33.33%;}  
.col-5 {width: 41.66%;}  
.col-6 {width: 50%;}  
.col-7 {width: 58.33%;}  
.col-8 {width: 66.66%;}  
.col-9 {width: 75%;}  
.col-10 {width: 83.33%;}  
.col-11 {width: 91.66%;}  
.col-12 {width: 100%;}
```

- Cross-browser: garantir a compatibilidade com todos os dispositivos e com diferentes browsers. Muitas vezes temos de implementar código diferente para diferentes browsers.
- Responsive Web Design: O layout do website deve adaptar-se automaticamente aos diferentes tamanhos de ecrã dos dispositivos; Muitas vezes temos de mudar o layout da página ou remover texto/images;
- Performance: Páginas Web devem ser rápidas carregar. Escolher tamanho ideal de imagens; minimizar o código; etc.
- Accessibility: Website deve ser acessível a todos - adaptar conteúdo a pessoas com deficiências visuais, deficiências auditivas, incapacidades cognitivas ou incapacidades físicas.
- Internationalization
- Privacy & Security

- Usar elementos HTML de forma a fornecer significado estrutural e contextual para o conteúdo da página.
- Em vez de simplesmente usar elementos HTML com base na sua aparência visual, selecionar cuidadosamente os elementos que melhor descrevem a estrutura e conteúdo. - **HTML Semantic**
- Elementos que descrevem melhor o conteúdo: `<header>`, `<nav>`, `<article>`, `<section>`, `<footer>` - em vez de usar só `<div>` para todo o conteúdo
- SEO: mecanismos de busca utilizam a semântica HTML apropriados para entender melhor o contexto do conteúdo e melhor no posicionamento dos resultados de pesquisa
- Cada elemento tem um propósito semântico claro que ajuda na manutenção do website

Práticas Web - exemplo

<header>

<section>

<footer>

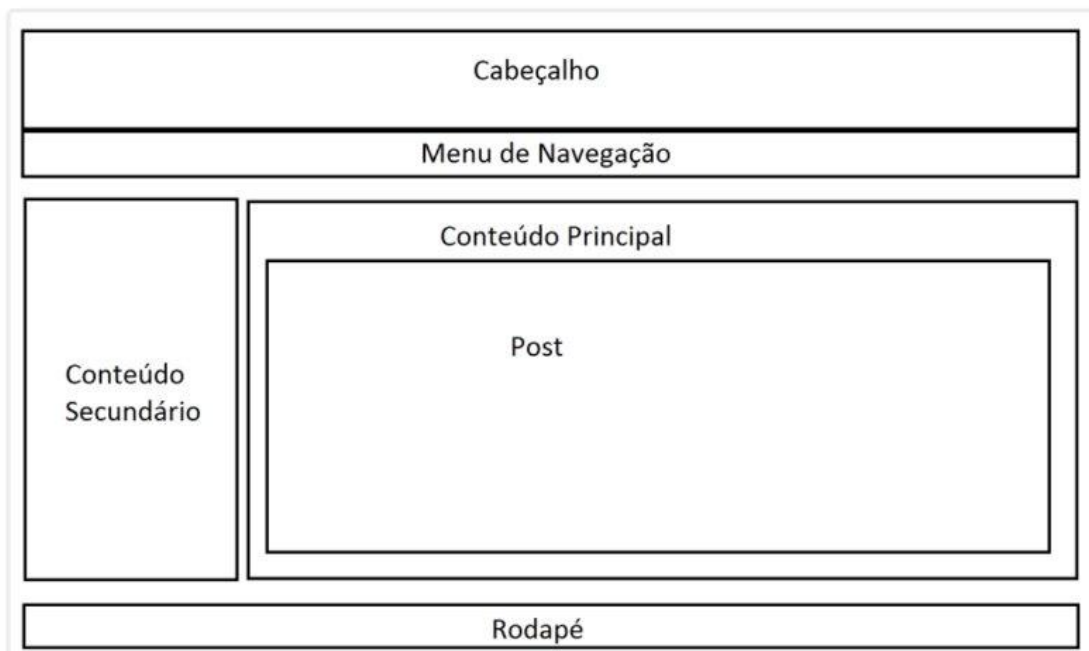


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exemplo de HTML Semântico</title>
</head>
<body>
  <header>
    <h1>Meu Site</h1>
    <nav>
      <ul>
        <li><a href="#">Início</a></li>
        <li><a href="#">Sobre</a></li>
        <li><a href="#">Contato</a></li>
      </ul>
    </nav>
  </header>

  <section>
    <article>
      <h2>Título do Artigo</h2>
      <p>Conteúdo do artigo...</p>
    </article>
  </section>

  <footer>
    <p>&copy; 2024 Meu Site. Todos os direitos reservados.</p>
  </footer>
</body>
</html>
```

Práticas Web

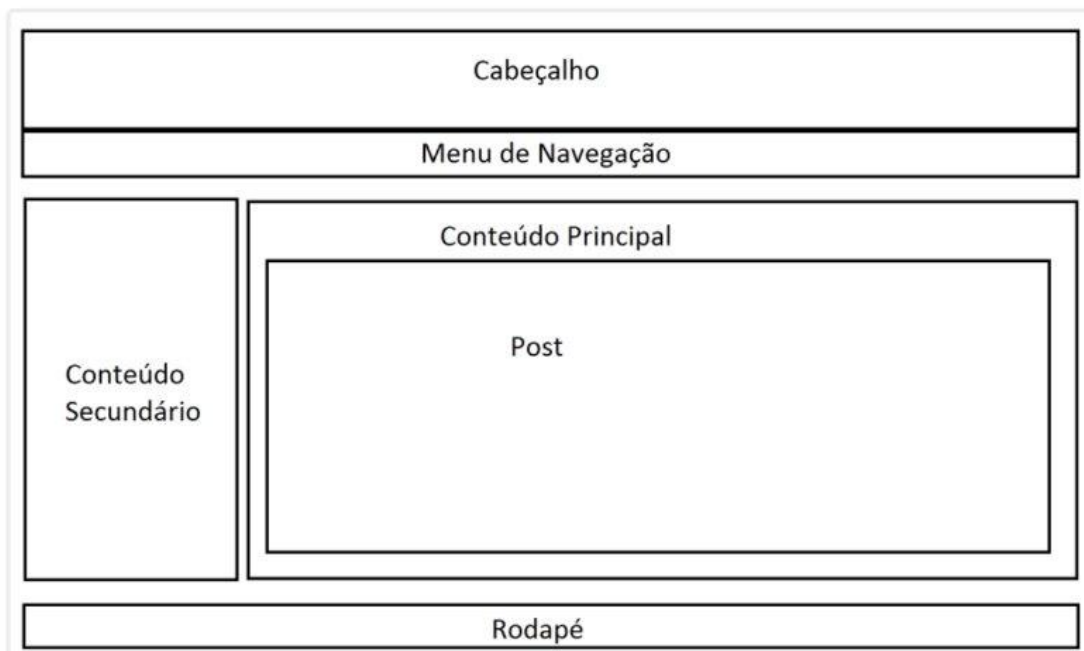


```
<div class="cabecalho" >
|   <div class="menu-navegacao" ></div>
</div>

<div class="conteudo-secundario" ></div>

<div class="conteudo-principal" >
|   <div class="post" ></div>
</div>

<div class="rodape" ></div>
```



```
<header class="cabecalho" >
|   <nav class="menu-navegacao" ></nav>
</header>

<aside class="conteudo-secundario" ></aside>

<main class="conteudo-principal" >
|   <article class="post" ></article>
</main>

<footer class="rodape" ></footer>
```

Práticas Web - Semântica

Div e Span: <div> não exibe nada, a menos que a usando CSS. As demais tags também são assim. Servem apenas para fazermos divisões.

- <div> é um elemento de bloco
- é um elemento de linha

Article

Define um artigo. Muito utilizado em Posts de blogs e sites de notícias.

Aside

Define um conteúdo ao lado do conteúdo da página. Muito utilizado em colunas ao lado da parte principal dos sites, onde normalmente possui a navegação, propagandas ou algo relacionado com o conteúdo principal.

Code

Define um pedaço de código de computador

Práticas Web - Semântica

Details e Summary

A tag <details> serve para acrescentar detalhes adicionais sobre algo, mas essa informação fica escondida, sendo exibida apenas quando o usuário clicar para ler.

A tag <summary> serve para exibir um título para o <details>, que ficará sempre visível.

Dialog

serve para criar modais (popups).

Footer

Define um rodapé para o documento ou seção.

Header

Define um cabeçalho para o documento ou seção.

Main

Define a parte principal de um documento. Por ser algo indicado como principal, o conteúdo dessa tag deve ser único e ela não deve ser filha de tags como <article>, <footer>, <header> ou <nav>.

Nav

Define um grupo de links de navegação. Essa tag é muito utilizada nos menus utilizados para se navegar entre as páginas de um site.

Section

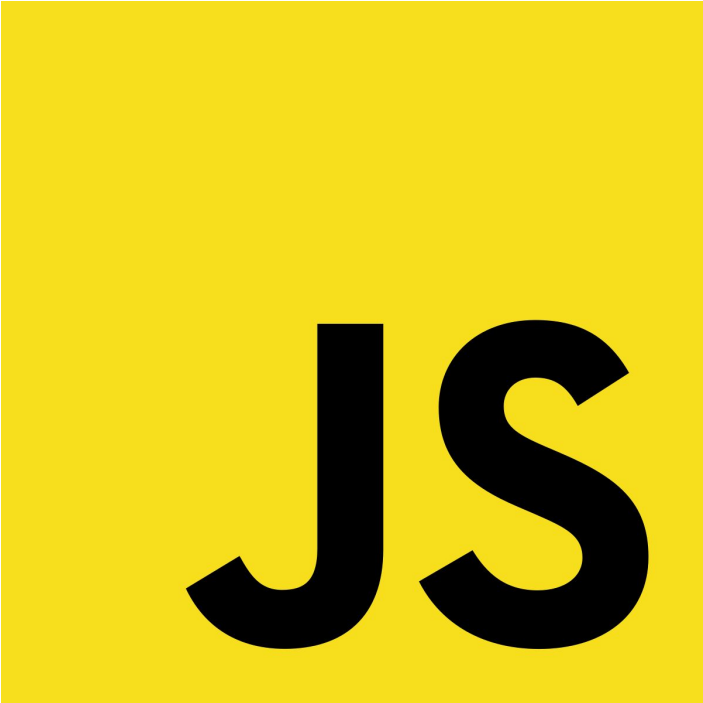
Funciona como a Div, sendo mais específica. Define seções no documento, como capítulos, cabeçalhos, rodapés, etc.

Javascript

Linguagem de programação, orientada a objetos, amplamente utilizada para desenvolver aplicações web.

Javascript permite manipular o conteúdo da página, responder a eventos, fazer chamadas assíncronas ao servidor, e interação.

Linguagem interpretada pelos navegadores Web - executado no lado do cliente



JS

Running and Loading JS

1. Adicionar código JS diretamente nos atributos de um elemento HTML:

```
<input type="button" value="click" onClick='alert("Button Clicked");' />
```

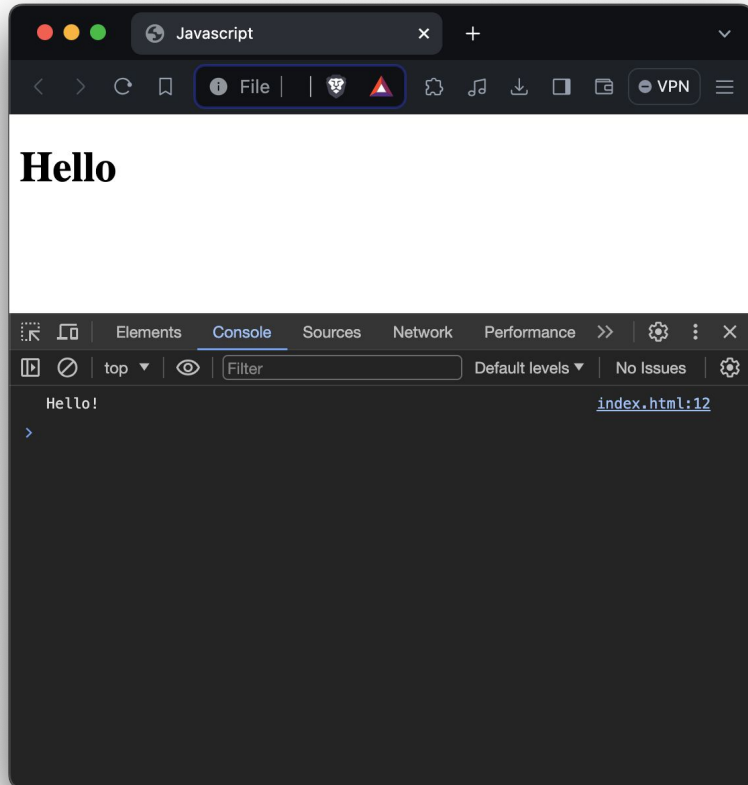
1. No ficheiro HTML, dentro de um elemento <script>:

```
<script type="text/javascript">  
    alert("Script Tag!!");  
</script>
```

1. Ficheiro Javascript:

```
<script src="./file.js"></script>
```

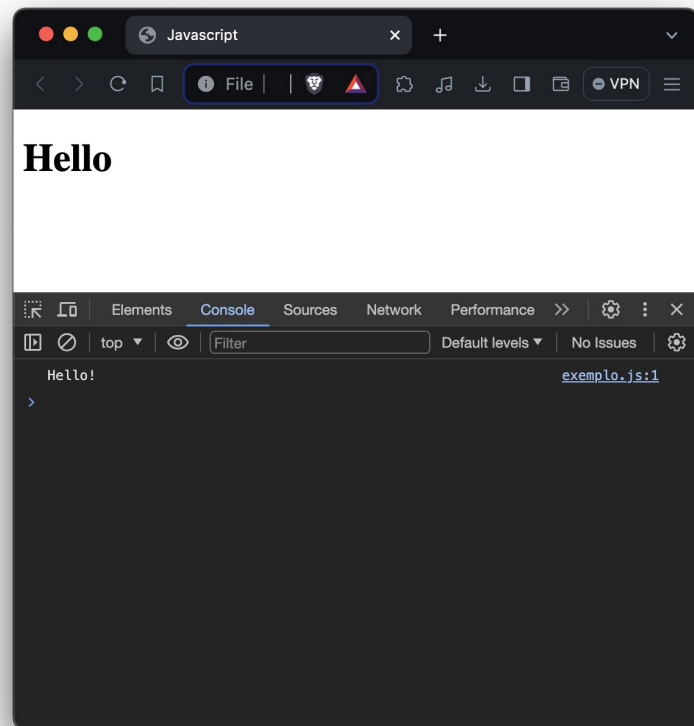
Running and Loading JS



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width
  <title>Javascript</title>
</head>
<body>
  <h1>Hello</h1>

  <script>
    console.log("Hello!");
  </script>
</body>
</html>
```

Running and Loading JS

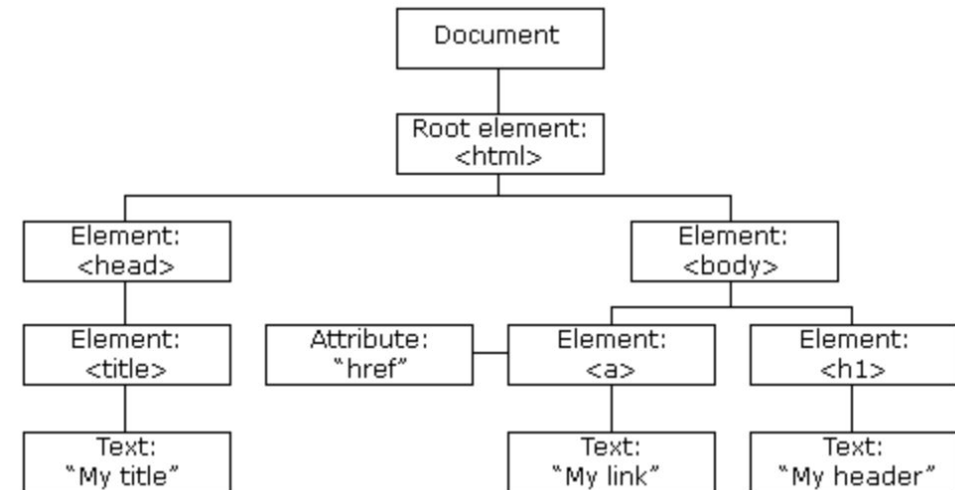


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-wi
  <title>Javascript</title>
</head>
<body>
  <h1>Hello</h1>

  <script src="exemplo.js"></script>
</body>
</html>
```

DOM - Document Object Model

- Representação em forma de árvore de todos os elementos de uma página Web.
- Objeto e interface para interagir e manipular os elementos HTML.
- Objeto que permite obter, alterar, adicionar ou eliminar elementos HTML;
- *document*.



DOM - Document Object Model

- Operações DOM:
 - Seleccionar elementos HTML;

```
let elementoHTML = document.getElementById('elementoID');
```

- Alterar ou manipular conteúdo;

```
elementoHTML.textContent = 'Novo texto';
elementoHTML.classList.add('nova_classe');
```

- Manipular eventos;

```
elementoHTML.addEventListener('click', function() {
  console.log('Clicou no elemento');
});
```

DOM - Document Object Model

- Operações DOM:
 - Alterar estilos CSS;

```
elementoHTML.style.color = 'red';
```

- Criar novos elementos HTML

```
let elementoHTML = document.createElement('div');  
document.body.appendChild(elementoHTML);
```

Manipular elementos HTML

```
<div class="menu">
  <h1 id='elementoID'>Titulo</h1>
  <p>Fotografias</p>
  <p>Música</p>
</div>
```

```
// obter elemento pelo seu ID
let elementoHTML = document.getElementById('elementoID');

// obter lista de elementos pela class:
let elementoHTML = document.getElementsByClassName('elementos');

// obter lista de elementos pela tag:
let elementoHTML = document.getElementsByTagName('p');

// seleccionar elemento por classe ou ID:
let elementoHTML = document.querySelectorAll('.class_elementos');

let elementoHTML = document.querySelector('#id_elemento');
```

Alterar estilos de elementos HTML

- Depois de termos acesso ao elemento específico, podemos manipular os seus estilos, classes, etc.
- A propriedade *style* permite-nos aceder ou alterar os estilos de um elemento.

```
<h1 id="titulo">Titulo</h1>
```

```
// obter elemento
let elemento_titulo = document.getElementById('titulo');

console.log(elemento_titulo.style.fontSize);

// alterar CSS atributos/valores
elemento_titulo.style.fontSize = "20px";
```

Manipular Eventos

- Existem diversos tipos de eventos que podem ser capturados e tratados em um página Web.
- Estes eventos podem invocar uma função associada.

Eventos de Mouse:

- **click**: users clica em um elemento;
- **dblclick**: user realiza um duplo clique em um elemento;
- **mouseover**: cursor do *mouse* entra em um elemento;
- **mouseout**: cursor do *mouse* sai de um elemento;

Eventos de Teclado:

- **keydown**: tecla do teclado pressionada;
- **keypress**: tecla é pressionada e largada;

Eventos de Formulário:

- **submit**: formulário submetido;
- **change**: valor de input do formulário alterado;

Eventos de Janela:

- **load**: ocorre após a janela ou elemento é carregado;
- **resize**: ocorre quando a janela do navegador é redimensionada;
- **scroll**: ocorre quando o conteúdo faz *scroll*;

Eventos de Tempo:

- **setTimeout**: função executada uma vez após um intervalo de tempo específico;
- **setInterval**: função executada repetidamente em intervalos de tempo específicos;

Manipular Eventos - exemplo

- Método `addEventListener()`
 - permite “anexar” um evento a um elemento HTML
 - podemos adicionar múltiplos eventos a um elemento

```
<h1 id="titulo">Titulo</h1>
```

```
let elemento_titulo = document.getElementById('titulo');

elemento_titulo.addEventListener('click', function() {
    console.log('Botão pressionado.');
```

Manipular elementos - cont.

Podemos seleccionar todos os elementos de uma classe através do método `.querySelectorAll`;
Para iterar sobre os elementos usamos o método *forEach*

```
<div class="class_div">Elemento 1</div>  
<div class="class_div">Elemento 2</div>  
<div class="class_div">Elemento 3</div>
```

```
let elementos = document.querySelectorAll('.class_div');  
  
// remover class dos elementos  
elementos.forEach(function(elemento) {  
    elemento.classList.remove('class_div');  
})
```

Manipular elementos - cont.

```
<div class="class_div">Elemento 1</div>  
<div class="class_div">Elemento 2</div>  
<div class="class_div">Elemento 3</div>
```

```
let elementos = document.querySelectorAll('.class_div');  
  
// adicionar class dos elementos  
elementos.forEach(function(elemento) {  
    elemento.classList.add('class_div');  
})
```

Criar elementos

- Podemos criar elementos HTML de forma dinâmica;
- Prática comum em Javascript quando existem dados dinâmicos ou interações do *user*.

```
<div id="container"></div>
```

```
let container = document.querySelector('#container');

// criar um novo elemento <p>
let paragrafo = document.createElement('p');

// Definir o conteúdo
paragrafo.textContent = 'Parágrafo criado dinamicamente!';

// Adicionar class ao elemento criado
paragrafo.classList.add('nova_class');

// Adicionar/anexar o elemento <p> ao <div>
container.appendChild(paragrafo);
```

Criar elementos

- Também podemos criar a nossa estrutura dentro de uma string com interpolação de variáveis ou expressões:

```
let nome = 'João';
let idade = 20;

let html = `
  <div class="user">
    <p>Nome: ${nome}</p>
    <p>Idade: ${idade}</p>
  </div>
`;

let container = document.getElementById('container');

// Inserir HTML ao elemento selecionado
container.innerHTML = html;
```

Exercício

1. Adicionar um evento ao elemento `h1`, para mudar a cor para vermelho ao clicar. Nota: usar método `toggle`
1. Adicionar um evento ao elemento `div`, que muda a cor de background quando o cursor para por cima.
Nota: usar evento: `mouseover` e `mouseout`;
1. Somar dois números. `<input>`
2. Mudar cor de elementos com a mesma class.

Mudar cor do Titulo ao 'click'

Mudo a cor quando o cursor passa por cima.

Somar dois números inteiros

Mudar cor individual de cada elemento com a mesma Classe

Elemento 1

Elemento 2

Elemento 3

Aceder parâmetros no URL

- Uma das formas de passar informação entre páginas é definir parâmetros no URL.

`https://iscte-iul.pt/about.html?nome=Antonio&apelido=Santos`

- A estrutura desses parâmetros é a seguinte:

```
?campo1=valor1&campo2=valor2&campo3=valor3
```

Aceder parâmetros no URL

- Para aceder a cada um dos valores, podemos usar o *URLSearchParams*

`https://iscte-iul.pt/about.html?nome=Antonio&apelido=Santos`

```
let parametros= new URLSearchParams(window.location.search)

console.log(parametros.get("nome"))
>> "Antonio"

console.log(parametros.get("apelido"))
>> "Santos"
```

- Espaço em memória que guarda um valor de um determinado tipo
- Características de uma variável:
 - Tipo
 - Nome
 - Valor
- Em Javascript, ao definir uma variável estamos a indicar o seu nome e o seu valor inicial. O tipo é inferido pelo valor.

Variáveis

- Ao contrário de outras linguagens de programação, em Javascript o tipo da variável é flexível, o que nos permite desenvolver o código sem ter de pensar e atribuir um tipo ao definir a variável;
- Usamos o operador de atribuição (=) para definir ou alterar o valor das variáveis;

```
let a = 8;  
let b = 'texto';  
let c = 2;  
  
a = c + 2;  
  
// a = 4
```

Tipos de dados

- Definem as operações que podem ser feitas nos dados, o significado dos dados e a maneira como os valores desse tipo são armazenados.
- Existem diferentes tipos de dados:
 - Undefined: variável não foi definida
 - Null: variável definida mas não tem nenhum valor atribuído;
 - Numbers;
 - Strings;
 - Booleans;
 - Arrays;
 - Object

```
let inteiro = 8;
let texto = "ola";
let lista = [1, 2, 3];
let objeto = {
  nome: "João",
  idade: 30
};
```

Instruções e Blocos de Instrução

- Instrução: ação que pode mudar o estado de uma variável. Em Javascript, uma instrução termina sempre com um ponto-e-vírgula (;)
- Um bloco de instruções é um conjunto de instruções entre chavetas que será executado sequencialmente

```
let inteiro = 8;
```

```
{
  a = b + 2;
  b = b - 1;
}
```

Operadores Matemáticos

- Cálculo entre dois valores

+	Soma
-	Subtração
*	Multiplicação
/	Divisão

+=	Adiciona valor a ele próprio
++	Adiciona 1 valor
--	Subtrai 1 valor

Operadores Comparação

- Comparar se duas ou mais variáveis se comparam

===	é igual?
<	é menor?
<=	é menor ou igual?

!=	é diferente?
>	é maior?
>=	é maior ou igual?

Operadores Lógicos

&&	conjunção (e / and)
	disjunção (ou / or)
!	negação

Operadores - Exemplos

```
2 === 2 -> true
```

```
2 === 3 -> false
```

```
2 !== 3 -> true
```

```
2 < 3 -> true
```

```
2 >= 3 -> false
```

```
true && true -> true
```

```
true && false -> false
```

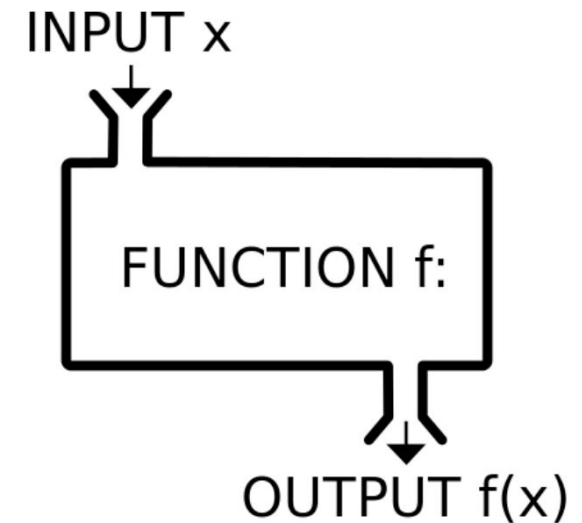
```
true || true -> true
```

```
false || true -> true
```

```
false || false -> false
```

Funções

- Bloco de instrução que performa uma tarefa. Essa tarefa só é executada se a função for chamada.
- Permite que blocos de código sejam reutilizados
- Pode ter argumentos - valores que a função usa quando é chamada
- Após a execução da tarefa pode retornar o resultado



Tipos de Funções

- Native:
 - Funções que fazem parte do motor Javascript (built into)
 - Ex: Math.max(10, 12), alert("test"), Date(), etc.

- Custom:
 - Funções criadas por nós para executar determinada tarefa

```
function square(x) {  
    return x * x;  
}
```

Funções

Declaração da Função:

```
function square(x) {  
    return x * x;  
}
```

Nome

Parâmetros

Corpo

Invocar a Função:

```
square(3);
```

Argumentos

```
return x * x;
```

return – indica qual o valor devolvido pela função

Funções - variáveis como argumentos

- O valor de um argumento pode ser dado usando uma variável;
- O argumento será o valor guardado na variável no momento em que a função é invocada.

```
function somar(x, y) {  
  return x + y;  
}
```

```
> let a = 5;  
> let b = somar(a, 3);  
  
>> b = 8
```

Funções - expressões como argumentos

- O valor de um argumento pode ser dado usando uma expressão;
- O argumento será o valor da expressão no momento em que a função é invocada

```
> let a = 5;  
> let b = somar(a - 2, 3);  
  
>> b = 6
```

Funções - resultado de funções como argumentos

- O valor de um argumento pode ser o valor devolvido por uma função;
- As invocações usadas como argumentos são executadas primeiro, de modo que o valor devolvido possa ser utilizado como argumento

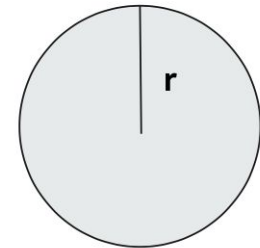
```
> let a = 5;
> let b = somar(subtrair(a, 2), 3);

>> b = 6
```

Exercício

1. Criar função que recebe 3 argumentos e devolve a soma dos dois primeiros, multiplicando depois o terceiro argumento;
2. Criar uma função que retorna a área de um círculo;
1. Criar função que converte a temperatura de graus Celsius para Fahrenheit;

$$\text{Área} = \pi r^2$$



$$F = \left[\frac{9}{5} C \right] + 32$$

Instrução if-else

- Permite condicionar a execução de uma ou várias instruções em função de determinada condição booleana (true/false)

```
if (condição) {
    executa código se condição é verdadeira
} else {
    executa código se condição é falsa
}
```

```
function min(a, b) {
    if (a < b) {
        return a;
    } else {
        return b;
    }
}
```

Instrução if-else

- Possível encadear várias condições para verificar várias condições:

```
if (condição_1) {
    executa código se condição_1 é verdadeira
} else if (condição_2) {
    executa código se condição_2 é verdadeira
}
```

Strings

- Em Javascript uma string é delimitada por aspas duplas (“ ”), simples (‘ ’) ou acentos graves (` `)

```
let string1 = “Ana”;  
let string2 = ‘Joana’;  
let string3 = `Mariana`;
```

Strings - propriedades e métodos

1. Tamanho da String

```
let string1 = "Ana";  
> console.log(string1.length)  
>> 3
```

2. Concatenação:

```
let string1 = "hello ";  
let string2 = "world!";  
string1.concat(string2);  
>> "hello world!"  
  
string1 + "everyone!";  
>> "hello everyone!"
```

3. Comparação: (case-sensitive)

```
let string1 = "hello world";  
let string2 = "HELLO WORLD";  
  
string1 === string2;  
>> false
```

Strings - propriedades e métodos

4. Interpolação

Permite incorporar valores ou expressões dentro de string. String tem se definida dentro de acentos graves (` `), para inserir valor de uma expressão:

```
let nome = "Ana";  
let idade = 30;  
let template = `A ${nome} tem ${idade} anos`;  
  
>> "A Ana tem 30 anos"
```

Exercício

1. Criar uma função que devolve:
 - Imprime “Está gelado”, se a temperatura for **inferior a 0**;
 - Imprime “Está frio”, se a temperatura for **inferior a 15**;
 - Imprime “Está quente”, se a temperatura for **superior a 50**;
 - Imprime “Está a ferver”, se a temperatura for **superior a 80**;
 - Imprime “Está normal”, para os outros casos.

Exercício

- Criar uma função que recebe o nome e a idade de uma pessoa e devolve uma String construída com uma mensagem que diz o ano que nasceu:
 - Ex: "Olá José, nasceste em 1993"

Strings (cont.)

- Podemos olhar para uma String como uma **sequência de caracteres**, e isso permite-nos obter o caracter específico numa certa posição:

```
let texto = `O José nasceu em 1993`;  
console.log(texto[2])  
  
>> "J"
```

Strings (cont.)

5. **Slice:** remove parte da string e devolve a parte extraída:

```
let texto = `Hello world!!`;
let result = texto.slice(0, 5);

>> "Hello"
```

start - primeira posição

end - posição onde
termina mas não
incluída no *split*

Strings (cont.)

6. **split**: divide a string em um array de substrings. Retorna o array criado e não altera a string inicial:

```
let texto = `Hello world!!`;
let result = texto.split(" ");

>> ["Hello", "world!!"]
```

separator - expressão
para efetuar o *split*

limit - limitar o
número de *splits*

Objetos

- Permite guardar diferentes informações e tipos de forma estruturada
- Os valores são associados a uma determinada chave/key

```
> console.log(objeto.nome)
>> "João"

> console.log(objeto.idade)
>> 30
```

Obter valor do campo

```
let objeto = {
  nome: "João",
  apelido: "Matos",
  idade: 30
};
```

Objetos

```
let objeto = {  
  nome: "João",  
  apelido: "Matos",  
  idade: 30  
}
```

```
// Adicionar um valor ao objeto  
objeto.altura = 190;
```

```
// Alterar um valor  
objeto.idade = 37;
```

```
// Adicionar, usando uma String como chave  
objeto['altura'] = 190;
```

```
// usar o valor de uma variável como chave  
let chave = idade;  
objeto[chave] = 37;
```

Objetos

```
let objeto = {  
  nome: "João",  
  apelido: "Matos",  
  idade: 30  
}  
  
// Obter chaves  
> console.log(Object.keys(objeto));  
>> ["nome", "apelido", "idade"]  
  
// Obter valores  
> console.log(Object.values(objeto));  
>> ["João", "Matos", 30]
```

```
// Remover chave / valor  
delete objeto.apelido;
```

Arrays

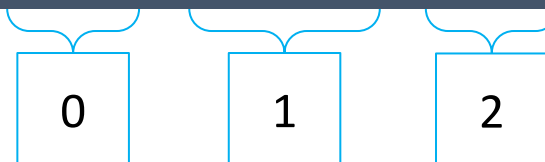
- Arrays (ou listas) permitem guardar uma quantidade indeterminada de informações em série;
- Podemos adicionar e aceder elementos através da posição em que eles estão

```
let carros = ["BMW", "Honda", "Audi"];
```

Arrays

- Aceder ao elemento através do índice:

```
let carros = ["BMW", "Honda", "Audi"];
```



```
console.log(carros[1]);
```

```
>> "Honda"
```

Arrays - tamanho

```
let carros = ["BMW", "Honda", "Audi"];
```

```
// obter tamanho do Array:  
console.log(carros.length);
```

```
>> 3
```

```
let carros = ["BMW", "Honda", "Audi"];
```

```
// aceder ao último elemento do Array:  
console.log(carros[carros.length - 1]);
```

```
>> "Audi"
```

Arrays - adicionar elemento

```
let carros = ["BMW", "Honda", "Audi"];

// adicionar um elemento:
carros.push("Mercedes");

>> ["BMW", "Honda", "Audi", "Mercedes"];
```

Arrays - encontrar index

```
let carros = ["BMW", "Honda", "Audi"];  
  
// encontrar posição/index do elemento:  
console.log(carros.indexOf("Honda"));  
  
>> 1
```

Arrays - método slice

```
let carros = ["BMW", "Honda", "Audi"];
```

```
console.log(carros.slice(1));
```

```
>> ["Honda", "Audi"]
```

Exercício

- Criar uma função que recebe um objeto com as propriedades **nome** e **anoNascimentos** e devolve o objeto com o **nome** e **idade**

```
let objeto = {  
  nome: "Francisco",  
  anoNascimento: 1998  
}  
  
function converterIdade(objeto){  
  ...  
}  
  
>> {  
  nome: "João",  
  idade: 26  
}
```

Exercício

Criar uma função que devolve um nome aleatório:

Dica: o método `Math.random()` devolve um valor aleatório de 0 a 1, nunca chegando a sair de 1; e `Math.floor()` retira as casas decimais de um número.

```
let nomes = ["João", "Maria", "Teresa"];
let sobrenomes = ["Silva", "Santos", "Carvalho"];

function nomeAleatorio(nomes, sobrenomes){
  ...
}

nomeAleatorio(nomes, sobrenomes);

>> "Teresa Silva"
```

Exercício

Criar uma função que devolve um nome aleatório:

Dica: o método `Math.random()` devolve um valor aleatório de 0 a 1, nunca chegando a sair de 1 ; e `Math.floor()` retira as casas decimais de um número.

```
function nomeAleatorio(nomes, sobrenomes){  
  console.log(`  
    ${nomes[Math.floor(Math.random() * 3)]} ${sobrenomes[Math.floor(Math.random() * 3)]}  
  `)  
}
```

- Para implementar um código flexível que consiga aceder aos elementos de um Array requer a utilização de **ciclos**. Podemos usar o mesmo código repetidamente para diferentes valores
- Como vimos, Arrays podem conter dimensões variadas.
- `let nomes = ["João", "Maria", "Teresa"];`

```
console.log(nomes[0]);  
>> "João"  
  
console.log(nomes[1]);  
>> "Maria"  
  
console.log(nomes[2]);  
>> "Teresa"
```

```
for (let i = 0; i < nomes.length; i++) {  
    console.log(nomes[i]);  
}  
  
>> "João"  
>> "Maria"  
>> "Teresa"
```

Ciclo *While*

- Permite correr o mesmo bloco várias vezes, enquanto uma determinada condição se mantiver;
- Código dentro do ciclo *while*, é corrido enquanto o valor na variável “contador” for superior a zero.

```
let contador = 5;
while (contador > 0) {
  console.log(contador);
  contador = contador - 1;
}
```

```
>> 5
>> 4
>> 3
>> 2
>> 1
```

Ciclo *While*

- Podemos indicar uma condição que é sempre verdadeira (“true”) e para o ciclo com uma condição interna através da instrução “**break**”;

```
let contador = 5;
while (true) {
  console.log(contador);
  contador = contador - 1;
  if (contador === 0)
    break;
}

>> 5
>> 4
>> 3
>> 2
>> 1
```

Ciclo *For*

- Sintaxe alternativa ao **while**, mais organizada para iterações em Arrays.

```
for (instrução_1; instrução_2; instrução_3)
```

- `instrução_1`: executa uma vez antes do bloco de código;
- `instrução_2`: condição a verificar para executar o bloco de código;
- `instrução_3`: executada sempre depois do bloco de código.

```
for (let contador = 5; contador > 0; contador--) {  
    console.log(contador);  
}
```

```
>> 5           >> 4  
>> 3           >> 2  
>> 1
```

Ciclo For

Exemplos:

```
for (let i = 0; i < nomes.length; i++) {  
    console.log(nomes[i]);  
}
```

```
>> "João"  
>> "Maria"  
>> "Teresa"
```

```
for (let i = nomes.length - 1; i >= 0; i--) {  
    console.log(nomes[i]);  
}
```

```
>> "Teresa"  
>> "Maria"  
>> "João"
```

Exercício

- Imprimir na consola todos os números pares de 1 a 100, usando as duas estruturas de ciclos (**for** e **while**)

Nota: operador '%' retorna o resto de uma divisão.

Array de Objetos

- Um array pode ter elementos de tipos diferentes:

```
let array = ["João", 1, true, {}];
```

- Array de objetos é uma estrutura de dados que armazena uma coleção de objetos:

```
let array = [
  {
    nome: "João",
    idade: 20
  },
  {
    nome: "Teresa",
    idade: 23
  }
];
```

Exercícios

- Imprimir o nome dos alunos do array de objetos:

```
let alunos = [  
  {  
    nome: "João",  
    idade: 20  
  },  
  {  
    nome: "Teresa",  
    idade: 23  
  },  
  {  
    nome: "Maria",  
    idade: 25  
  }  
];
```

Funções como argumento

- Funções de **callback** operam como variáveis normais enviadas no argumento de outra função, e são invocadas por essa função:

```
function teste1(){
    return "Bom dia!";
}
imprimir(teste1);
```

```
function imprimir(funcao){
    console.log(funcao());
}
```

```
function teste2(){
    return "Hello World!";
}
imprimir(teste2);
```

Funções como argumento

```
function imprimirNvezes(n, funcao) {  
    for (let i = 0; i < n; i++) {  
        console.log(funcao(i));  
    }  
}
```

```
function teste1(num) {  
    return 2 ** num;  
}  
imprimirNvezes(10, teste1);
```

```
function teste2(num) {  
    return 3 ** num;  
}  
imprimirNvezes(10, teste2);
```

Método .forEach()

- Chama a função para cada elemento do array;
- Coloca o valor do elemento do array no primeiro parâmetro dessa função;
- Função executada é chamada de “callback”

```
let alunos = [
  {nome: "João", idade: 20},
  {nome: "Teresa", idade: 23},
  {nome: "Maria", idade: 25}
];

function imprimirNome(aluno){
  console.log(aluno.nome);
}

alunos.forEach(imprimirNome);
```

Funções anónimas

- Nem sempre é necessário declarar uma função isoladamente para o código a conseguir chamar.
- Uma função anônima é uma função que não tem nome, e pode ser declarada diretamente no *callback* do método ***forEach***

```
let alunos = [  
  {nome: "João", idade: 20},  
  {nome: "Teresa", idade: 23},  
  {nome: "Maria", idade: 25}  
];  
  
alunos.forEach(function (aluno) {  
  console.log(aluno.nome);  
});
```

Exercício

1. Cria um algoritmo que imprime o nome e a idade de cada aluno do array de objetos, utilizando o método **.forEach()**
1. Imprimir apenas os alunos que têm mais de 21 anos

```
let alunos = [  
  {  
    nome: "João",  
    idade: 20  
  },  
  {  
    nome: "Teresa",  
    idade: 23  
  },  
  {  
    nome: "Maria",  
    idade: 25  
  }  
];
```

Método .map()

- O método **.map()** itera um array e devolve uma lista nova, com o resultado devolvido pelo callback;
- Não altera o array inicial;

```
let alunos = [  
  {nome: "João", idade: 20},  
  {nome: "Teresa", idade: 23},  
  {nome: "Maria", idade: 25}  
];  
  
let nomes = alunos.map(function (aluno) {  
  return aluno.nome;  
});  
  
console.log(nomes);  
  
>> ["João", "Teresa", "Maria"]
```

Método .filter()

- O método **.filter()** devolve um array novo, com os elementos do array inicial que passaram numa função de validação;
- Não altera o array inicial;

```
let alunos = [  
  {nome: "João", idade: 20},  
  {nome: "Teresa", idade: 23},  
  {nome: "Maria", idade: 25}  
];  
  
let alunos_21 = alunos.filter(function (aluno) {  
  return aluno.idade > 21;  
});  
  
console.log(alunos_21);  
  
>> [{nome: "Teresa", idade: 23},  
  {nome: "Maria", idade: 25}]
```

Exercício

1. Escreve um algoritmo que transforma a lista à direita numa lista de nomes completos dos alunos utilizando a função map:

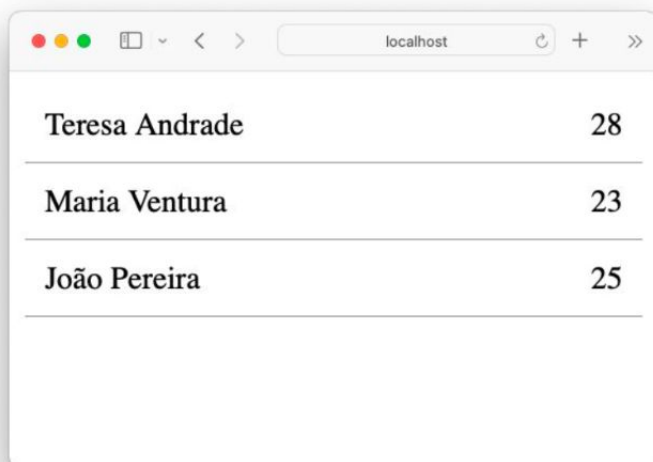
```
["João Andrade", "Teresa Ventura", "Maria  
Pereira"];
```

1. Usando o método filter(), imprime o array acima mas apenas os alunos com mais de 21 anos.

```
let alunos = [  
  {  
    nome: "João",  
    apelido: "Andrade",  
    idade: 20  
  },  
  {  
    nome: "Teresa",  
    apelido: "Ventura",  
    idade: 23  
  },  
  {  
    nome: "Maria",  
    apelido: "Pereira",  
    idade: 25  
  }  
];
```

Exercício

- Utilizando a seguinte lista de alunos, adicionar ao elemento div os dados do objeto alunos e apresentar da seguinte forma:



A screenshot of a web browser window displaying a table with three rows of student data. The browser's address bar shows 'localhost'. The table has two columns: the first column contains the full names of the students, and the second column contains their ages. The rows are: Teresa Andrade (28), Maria Ventura (23), and João Pereira (25).

Teresa Andrade	28
Maria Ventura	23
João Pereira	25

```
let alunos = [  
  {  
    nome: "Teresa",  
    apelido: "Andrade",  
    idade: 28  
  },  
  {  
    nome: "Maria",  
    apelido: "Ventura",  
    idade: 23  
  },  
  {  
    nome: "João",  
    apelido: "Pereira",  
    idade: 25  
  }  
];
```

```
<div class="lista_alunos"></div>
```

Exercício

- Em grupo, criar a página “Contact” com um formulário

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:3000/FullPageAula/contact.html?n...'. The website has a green header with the title 'Casas' and navigation links: 'Inicio', 'Sobre', 'Serviços', and 'Contacto'. The main content area is titled 'Contact page' and contains a form with the following fields:

- Nome:
- Email:
- Ano de nascimento:
- Cidade:

Below the form is a 'Submit' button. The form has been submitted, and the following information is displayed:

Nome: João Matos
Email: joao@gmail.com
Idade: 38
Cidade: Carcavelos

Below this, there is a table with the following data:

Teresa Andrade	28
Maria Ventura	23
João Pereira	25

The browser's address bar also shows a 'VPN' icon.

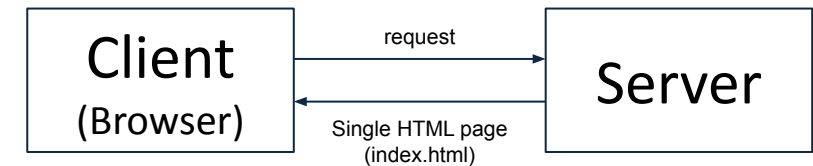
React

Biblioteca Javascript que permite desenvolver websites interactivos e *User Interfaces*

Permite criar *Single Page Apps* -> Improves Loading Speed

Vantagens:

- Reutilizar Components
- Virtual DOM: permite actualizações dos componentes HTML de forma rápida e eficaz;
- JSX: código HTML dentro de Javascript
- *State e Props*



React - componentes

- Componentes permitem separar a interface em diferentes “peças”
- Funções Javascript, que aceitam parâmetros (props) e retornam *markup*
- Componente é uma parte da interface (código) que uma lógica e aparência próprias
- Componentes podem ser chamados/reutilizados dentro de outros componentes
- Componente não pode fazer o return de múltiplos JSX elementos
- Existe pelo menos um componente principal, que corresponde à hierarquia máxima e todos os outros componentes serão descendentes.

```
function MyApp() {
  return (
    <button>Button!</button>
  );
}

export default function MyApp() {
  return (
    <div>
      <h1>My App!</h1>
      <MyButton />
    </div>
  );
}
```

React - componentes



Componente top_bar

Componente navigation

Componente slider

Apresentar dados

- Podemos usar Javascript dentro de markups:
`<h1> {user.name} </h1>`
- Dentro de {} podemos usar variáveis ou expressões Javascript

```
return (  
  <img  
    className="imagem"  
    src={user.imagemURL}  
  />  
);  
}
```

```
const user = {  
  name: 'John Doe',  
  age: 40,  
  bio: 'Lorem ipsum',  
};  
  
export default function MyApp() {  
  return (  
    <div>  
      <h1>{user.name}</h1>  
      <p>Age: {user.age}</p>  
      <p>Bio: {user.bio}</p>  
    </div>  
  );  
}
```

Listas

- Dentro dos componentes podemos usar métodos Javascript como ciclos for ou array map(), para apresentar listas de objetos ou arrays.
- Neste exemplo usamos map() para converter um array de objetos em um array de elementos HTML

Nota: Quando criamos uma lista temos de atribuir um id único a cada elemento da lista. O id vai servir de identificar para cada elemento.

```
const products = [  
  { title: 'Cabbage', isFruit: false, id: 1 },  
  { title: 'Garlic', isFruit: false, id: 2 },  
  { title: 'Apple', isFruit: true, id: 3 },  
];  
  
export default function ShoppingList() {  
  const listItems = products.map(product =>  
    <li key={product.id}>  
      {product.title}  
    </li>  
  );  
  return (  
    <ul>{listItems}</ul>  
  );  
}
```

Listas

```
const products = [  
  { title: 'Cabbage', isFruit: false, id: 1 },  
  { title: 'Garlic', isFruit: false, id: 2 },  
  { title: 'Apple', isFruit: true, id: 3 },  
];
```

```
export default function ShoppingList() {  
  const components = [];  
  
  products.forEach(product => {  
    components.push(  
      <Product  
        key={product.id}  
        title={product.title}  
      />  
    )  
  });  
  return (  
    <div>  
      {components}  
    </div>  
  );  
}
```

Eventos

Event handler functions:

1. Declarar função dentro do componente;
2. Implementar lógica dentro da função;
3. Adicionar evento e função;

Exemplo de eventos:

onClick ; onDrag ; onDoubleClick

onKeyDown ; onKeyPress

```
function MyButton() {  
  function handleClick() {  
    alert('You clicked me!');  
  }  
  return (  
    <button onClick={handleClick}>  
      Click me  
    </button>  
  );  
}
```

React: *state*

state: objeto que guarda variáveis,
informação, que pertence ao componente;

- Quando o objeto **state** muda/altera os seus valores o componente *re-renders*;

Import **useState**;

```
import { useState } from 'react';
```

Declarar a variável estado dentro do componente:

```
const [count, setCount] = useState(0);
```

```
import { useState } from 'react';

function MyButton() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <button onClick={handleClick}>
      Clicked {count} times
    </button>
  );
}
```

React: *state*

- O *state* e a lógica pertence dentro do componente.
- O componente que chama o componente Button, não precisa de saber a lógica que está implementada.
- Se chamarmos o componente várias vezes, cada elemento tem o seu próprio estado;

Clicked 3 times

Clicked 5 times

```
import { useState } from 'react';

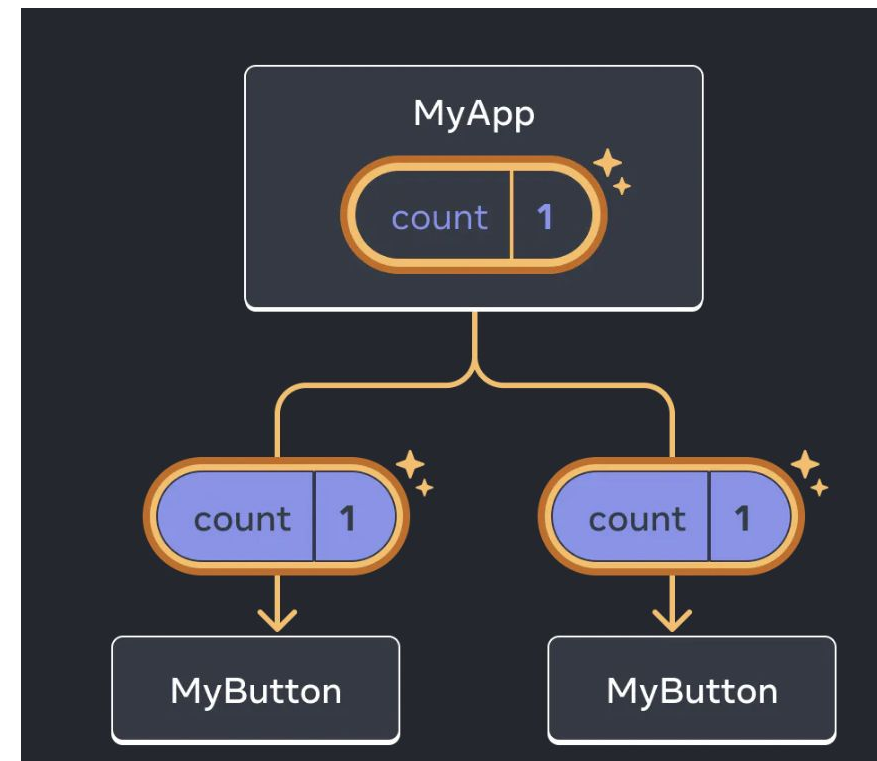
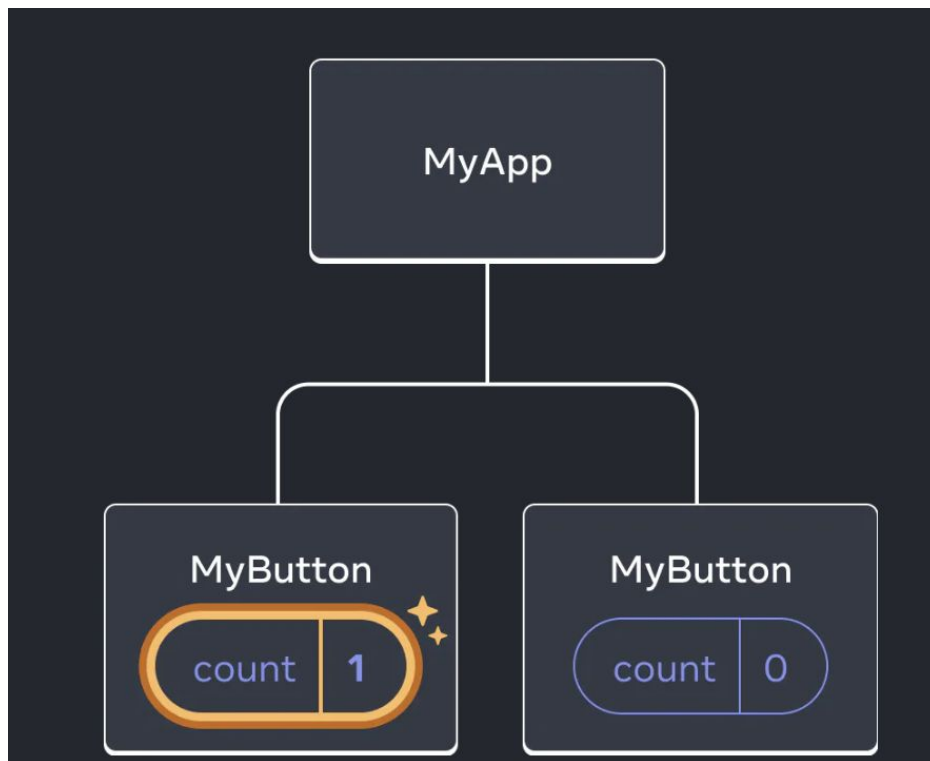
export default function MyApp() {
  return (
    <div>
      <MyButton />
      <MyButton />
    </div>
  );
}

function MyButton() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <button onClick={handleClick}>
      Clicked {count} times
    </button>
  );
}
```

React: compartilhar dados entre componentes



React: compartilhar dados entre componentes

```
import { useState } from 'react';

export default function MyApp() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <MyButton count={count} onClick={handleClick} />
      <MyButton count={count} onClick={handleClick} />
    </div>
  );
}
```

```
function MyButton({ count, onClick }) {
  return (
    <button onClick={onClick}>
      Clicked {count} times
    </button>
  );
}
```

Clicked 4 times

Clicked 4 times

React: props

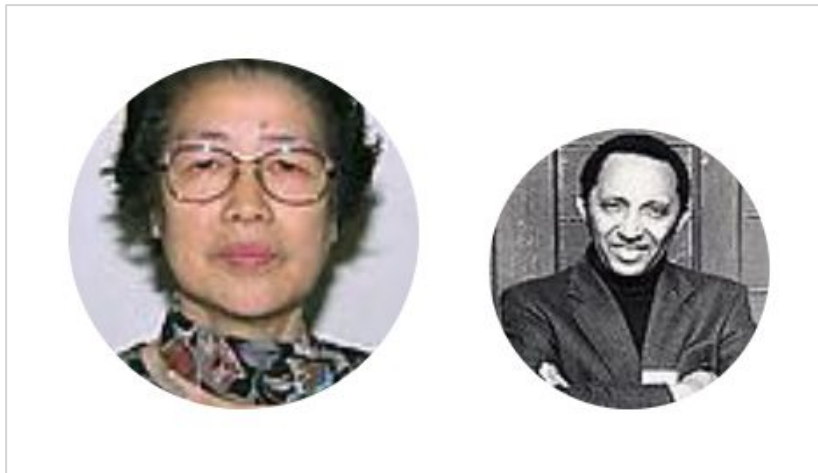
- *props* - propriedades
- Os componentes React usam para *props* “comunicar” entre eles
- Permite passar qualquer valor Javascript, como objetos, arrays, funções, etc.
- Componente Profile: passa 2 *props* (person e size) para o componente Avatar.

```
export default function Profile() {
  return (
    <div>
      <Avatar
        person={{ name: 'Lin Lanying',
                    imageId: '1bX5QH6' }}
        size={100}
      />
    </div>
  );
}
```

```
function Avatar({ person, size }) {
  return (
    <img
      src={getImageUrl(person)}
      alt={person.name}
      width={size}
      height={size}
    />
  );
}
```

React: props

- Podemos usar o componente múltiplas vezes, sem ter de adicionar o código novamente;



```
export default function Profile() {
  return (
    <div>
      <Avatar
        size={100}
        person={{
          name: 'Katsuko Saruhashi',
          imageId: 'Yfe0qp2'
        }}
      />
      <Avatar
        size={80}
        person={{
          name: 'Aklilu Lemma',
          imageId: 'OKS67lh'
        }}
      />
    </div>
  );
}
```

React: props

```
function Avatar({ person, size }) {  
  // ...  
}
```

```
function Avatar(props) {  
  let person = props.person;  
  let size = props.size;  
  // ...  
}
```

“Desconstruir” o objeto props: equivalente a ler os parâmetros de uma função.

```
function Profile(props) {  
  return (  
    <div className="card">  
      <Avatar {...props} />  
    </div>  
  );  
}
```

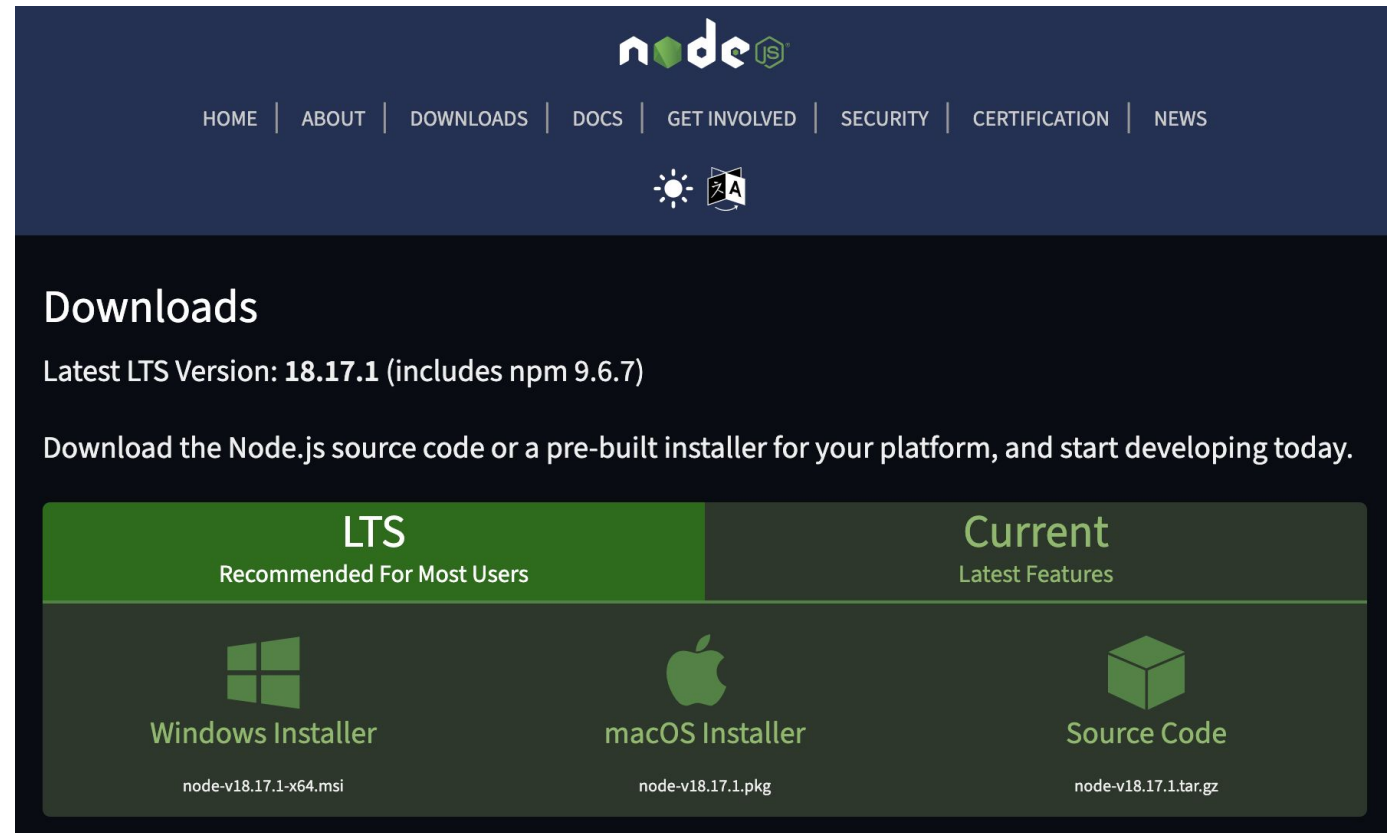
Podemos reencaminhar todos os props para o componente child

NodeJS: Download and install Node.js

Download Node.js from the official website:

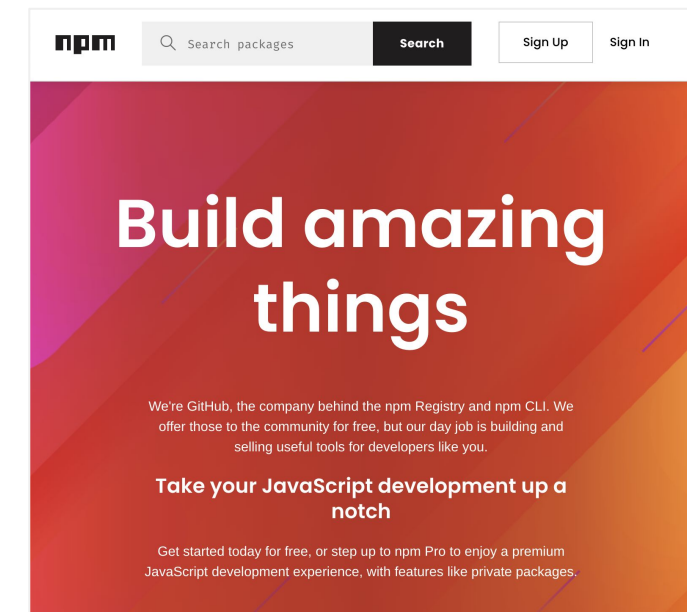
<https://nodejs.org/en>

Download and install the appropriate package for your system (Windows, macOS, Linux)



NPM - Node Package Manager

- Gestor de pacotes Node.js
- Disponibiliza uma linha de comandos para instalar, remover e listar módulos, assim como executar processors, etc.
- Podemos gerir as várias dependências do nosso projeto - garantir que todas as bibliotecas de que precisamos estão disponíveis
- O npm disponibiliza um enorme repositório de módulos já implementados e que podemos utilizar no nosso projeto



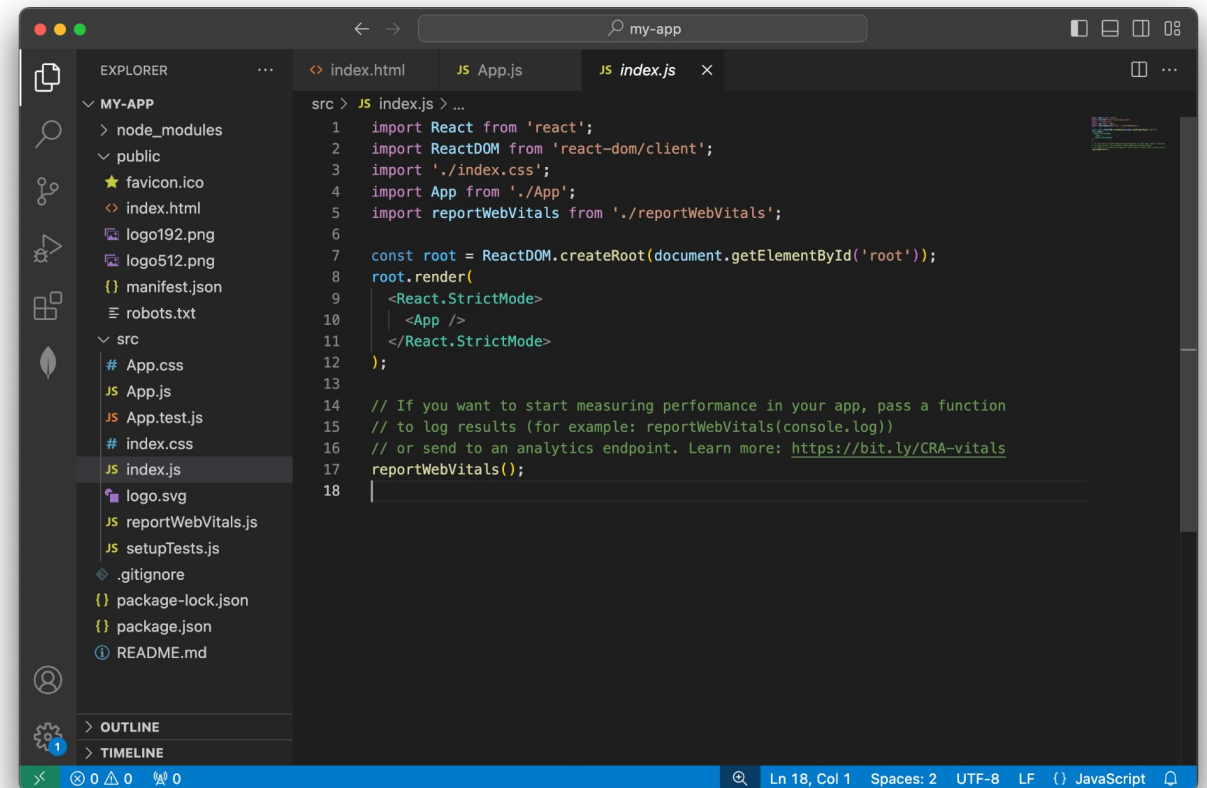
Package.json

- Ficheiro JSON
- Ficheiro de configuração de um projeto Node.js
- Descreve as dependências do projeto, scripts de execução, etc.
- `npm init`

```
{  
  "name": "react_e_fixe",  
  "version": "1.0.0",  
  "description": "Primeira App React",  
  "main": "index.js",  
  ▶ Debug  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "author": "",  
  "license": "ISC"  
}
```

Instalar React

- `npx create-react-app my-app`
- `npx`, comando do `npm` que instala e corre módulos diretamente, sem necessidade de fazer `npm install`

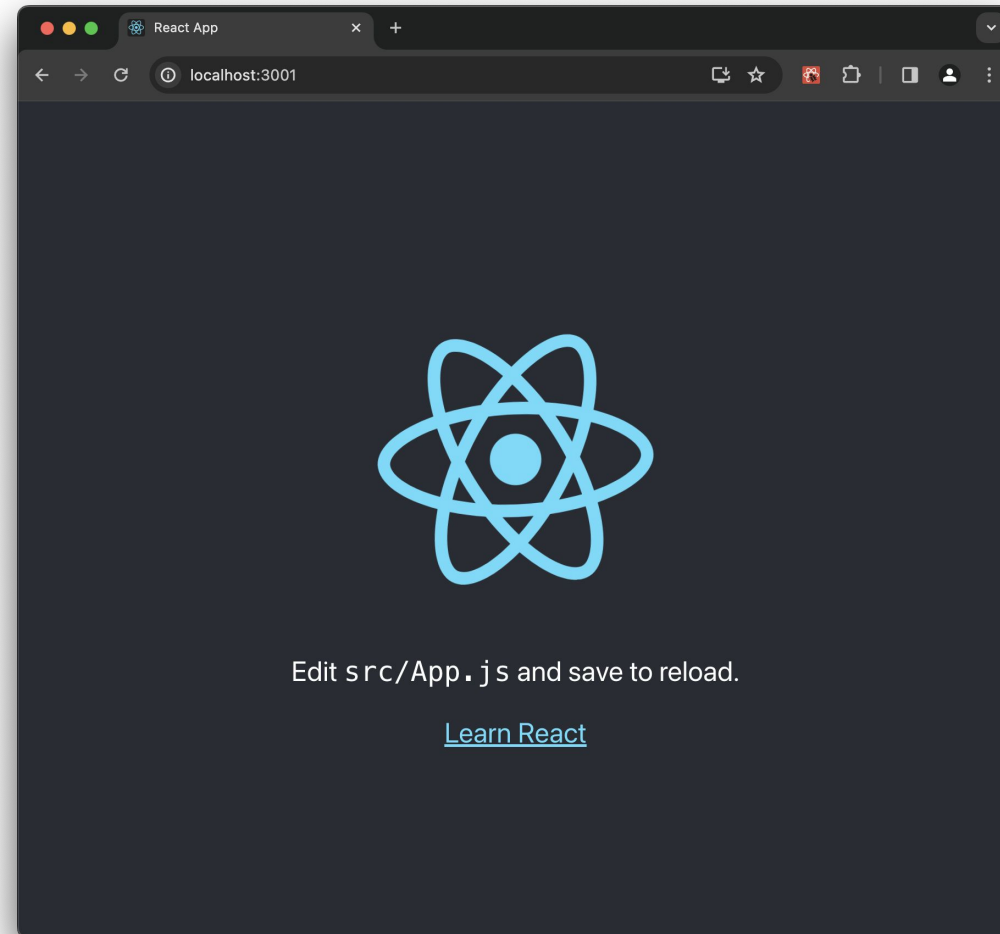


The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'MY-APP' with files like 'node_modules', 'public', 'index.html', 'logo192.png', 'logo512.png', 'manifest.json', 'robots.txt', 'src', 'App.css', 'App.js', 'App.test.js', 'index.css', 'index.js', 'logo.svg', 'reportWebVitals.js', 'setupTests.js', '.gitignore', 'package-lock.json', 'package.json', and 'README.md'. The code editor shows the content of 'index.js' with the following code:

```
src > JS index.js > ...
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import './index.css';
4  import App from './App';
5  import reportWebVitals from './reportWebVitals';
6
7  const root = ReactDOM.createRoot(document.getElementById('root'));
8  root.render(
9    <React.StrictMode>
10     <App />
11   </React.StrictMode>
12 );
13
14 // If you want to start measuring performance in your app, pass a function
15 // to log results (for example: reportWebVitals(console.log))
16 // or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
17 reportWebVitals();
18
```

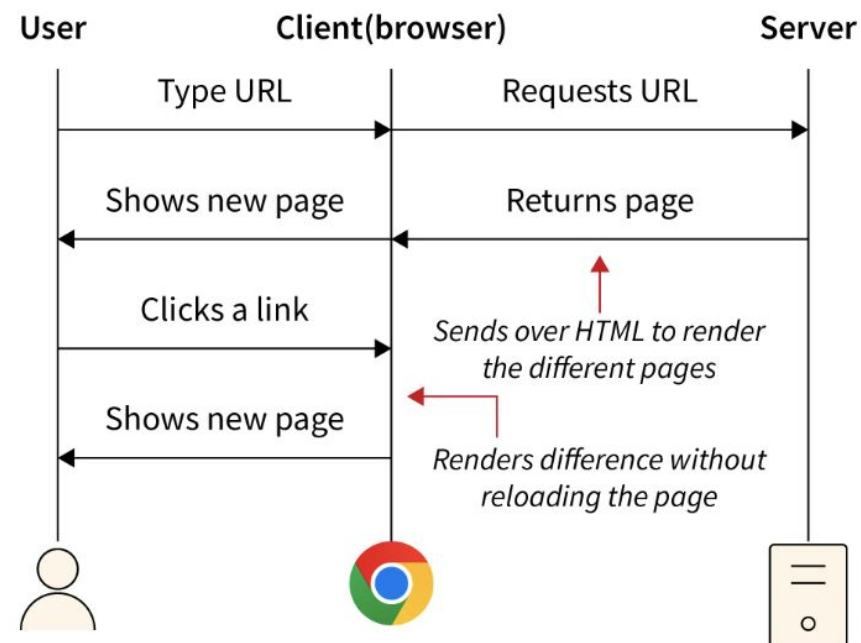
Run React

- `npm start`
- Local development server:
 - `http://localhost:3000`



React Router

- Quando a aplicação tem várias páginas/ecrãs, como podemos gerir a apresentação?
- React Router, permite-nos controlar a navegação entre páginas (Client side routing).
- Permite fazer o update do URL sem fazer outro pedido ao servidor



`npm install react-router-dom`

React Router

App

```
import { BrowserRouter as Router, Routes, Route } from
"react-router-dom";

import { Home } from "../pages/Home/Home";
import { Books } from "../pages/Books/Books";

function App() {
  return (
    <div className="App">
      <Router>
        <Header />
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/books" element={<Books />} />
        </Routes>
      </Router>
    </div>
  );
}
```

Navigation

```
import { NavLink } from "react-router-dom";

const Header = () => {
  return (
    <>
      <NavLink to="/" >
        Home
      </NavLink>
      <NavLink to="/books" >
        Books
      </NavLink>
    </>
  )
}
```

Link para página

```
<Link to="/books">Books</Link>
```

Eventos - revisão

Event handler functions:

1. Declarar função dentro do componente;
2. Implementar lógica dentro da função;
3. Adicionar evento e função;

Exemplo de eventos:

onClick ; onDrag ; onDoubleClick

onKeyDown ; onKeyPress

```
function MyButton() {  
  function handleClick() {  
    alert('You clicked me!');  
  }  
  return (  
    <button onClick={handleClick}>  
      Click me  
    </button>  
  );  
}
```

Eventos - revisão

- Quando um evento é gerado, a função que manipula esse evento tem acesso a um objeto *event*;
- Objeto event possui informação extra acerca do evento;
- Propriedade *target* (*event.target*)
 - devolve o elemento onde ocorreu o evento

```
function handleClick(event) {
  // Acesso ao elemento que disparou o evento
  console.log(event.target);
}

return (
  <button onClick={handleClick}>
    Click me
  </button>
);
```

```
function handleClick(event, var1) {
  // Acesso ao elemento que disparou o evento
  console.log(event.target);
  // Acesso ao argumento passado na função
  console.log(var1);
}

return (
  <button onClick={(event) => handleClick(event, 'variavel_1')}>
    Click me
  </button>
);
```

Propagação de eventos

- Manipuladores de eventos ou *event handlers*, também são executados se o evento ocorrer dentro de elementos *child*.
- O evento propaga-se até chegar ao elemento *parent* que está à espera de um evento.

```
function MyComponent() {
  function handleClick() {
    alert('You clicked me!');
  }

  return (
    <div onClick={handleClick}>
      <button onClick={() => alert('Button clicked!');}>
        Click me
      </button>
    </div>
  );
}
```

Parar propagação de eventos

- Manipuladores de eventos recebem como argumento um objeto evento, que possui informação acerca do evento;
- Objeto evento, usualmente denominado de 'e'
- Podemos usar este objeto para parar a propagação:
 `e.stopPropagation();`

```
function MyComponent() {
  function handleClick() {
    alert('You clicked me!');
  }

  return (
    <div onClick={handleClick}>
      <button onClick={(e) => {
        e.stopPropagation();
        alert('Button clicked!');
      }}>
        Click me
      </button>
    </div>
  );
}
```

Parar propagação de eventos

- Navegadores de internet também têm eventos ou comportamentos *default* associados a elementos;
- Por exemplo, ao submeter uma <form>, o navegador web faz o refresh automático da página;
- Podemos prevenir esse comportamento através do objeto evento (e)

e.preventDefault();

```
function MyComponent() {
  function submitForm(e) {
    e.preventDefault();
    alert('Submit form!');
  }

  return (
    <form onSubmit={submitForm}>
      <input/>
      <button>Send</button>
    </form>
  );
}
```

Evento onChange

- Evento que “escuta” uma alteração do valor de um elemento.
- Normalmente usado para lidar com mudanças de valor em elementos de formulário como, input, select, textarea;

```
export default function App() {  
  const [userName, setUsername] = useState('');  
  
  function handleChangeName(e) {  
    setUsername(e.target.value);  
  }  
  
  return (  
    <input  
      value={userName}  
      onChange={handleChangeName}  
    />  
  );  
}
```

Evento onChange

- Evento que “escuta” uma alteração do valor de um elemento.
- Normalmente usado para lidar com mudanças de valor em elementos de formulário como, input, select, textarea;

```
export default function App() {  
  
  const [firstName, setFirstName] = useState('');  
  const [lastName, setLastName] = useState('');  
  
  function handleFirstNameChange(e) {  
    setFirstName(e.target.value);  
  }  
  
  function handleLastNameChange(e) {  
    setLastName(e.target.value);  
  }  
  
  return (  
    <>  
    <input  
      value={firstName}  
      onChange={handleFirstNameChange}  
    />  
    <input  
      value={lastName}  
      onChange={handleLastNameChange}  
    />  
    </>  
  );  
}
```

State: modificar objetos

- Objetos não devem ser modificados;
- Por norma, deve-se criar ou copiar o objeto inicial. Modificar o objeto copiado e depois alterar o objeto de estado.
- No exemplo, setUser({name: "Doe"}), todos os outros campos do objeto vão ser apagados;
- Quando o estado é alterado, todo o estado é *overwritten*.

```
function MyComponent() {
  const [user, setUser] = useState({
    name: "John",
    age: 30,
    email: "john_doe@example.com"
  });

  function handleNameChange(e) {
    setUser({name: e.target.value});
  }

  (...)
}
```

State: modificar objetos

1. Fazemos uma cópia do estado anterior;
2. Modificamos os campos necessário;

Usando operador de propagação (...):

```
setUser({
  ...user,
  name: e.target.value
});
```

Copia todos os campos anteriores

Altera apenas (override) o valor do campo name

```
function MyComponent() {
  const [user, setUser] = useState({
    name: "John",
    age: 30,
    email: "john_doe@example.com"
  });

  function handleLastNameChange(e) {
    setUser({
      ...user,
      name: e.target.value
    });
  }

  (...)
}
```

State: modificar objetos

- Como fazemos quando temos múltiplos *inputs*?
- Múltiplos manipuladores de evento?

```
function handleChange(e) {  
  setUser({  
    ...user,  
    [e.target.name]: e.target.value  
  });  
}
```

State: modificar arrays

- Arrays não devem ser modificados;
- Por norma, deve-se criar ou copiar o array inicial. Modificar o array copiado e depois alterar o estado.
- Array métodos como push, pop, shift, etc. devem ser evitados;
- Usar concat, [...arr], filter, map.

```
(...)  
  
function handleAddUser() {  
  setUsers([  
    ...users,  
    { id: nextId++, name: userName }  
  ]);  
}  
  
(...)
```

Exercício

- Criar um novo componente;
- Criar uma lista com o nome de cada user do array;
- Adicionar novo objeto ao array;

```
export default function Component() {

  const [newUser, setNewUser] = useState('');

  const [users, setUsers] = useState([
    {id: 0, name: "John"},
    {id: 1, name: "Doe"},
  ]);

  function handleChangeInput(e) {
    (...)
  }

  function handleAddUser() {
    (...)
  }

  return (
    <>
      <input value={userName} onChange={handleChangeInput} />
      <button onClick={handleAddUser}>Add</button>
      <ul>
        { /* Implementar método que retorna lista de nomes */ }
      </ul>
    </>
  )
}
```

useEffect

- Hook que permite configurar determinados comportamentos a partir de mudanças no estado do componente;
- Recebe um callback que pode ser “chamado” quando, por exemplo:
 - O componente é criado; alterado ou removido.

```
useEffect(() => {  
    console.log("componente criado")  
}, [variavel1, variavel2]);
```

- As variáveis permitem dizer ao *useEffect* para correr quando algum destes estados é alterado

useEffect

- Permite chamar funções dentro do componente:
 - Fetch data; alterar estado; modificar a DOM

```
useEffect(() => {  
  console.log("componente criado")  
}, []);
```

```
useEffect(() => {  
  console.log("componente criado")  
  return function () {  
    console.log("componente removido")  
  }  
}, []);
```

```
useEffect(() => {  
  console.log("componente atualizado")  
});
```

```
useEffect(() => {  
  console.log("estado atualizado")  
}, [estado]);
```

```
useEffect(() => {  
  // Code that requires cleanup...  
  return () => {  
    // Cleanup code...  
  };  
}, [variavel]);
```